

Programming Assignment 01 Written Analysis

Question 1.

My “steps” heuristic is comparing the number of mismatched characters between the current state and the goal. The minimum number of actions needed to reach the goal is the same number. This is because each can only change a single letter. Therefore, it gives a lower bound for the cost. The true optimal cost couldn’t be greater than the number of mismatches, so the cost couldn’t be overstated.

Pseudocode:

```
num_mismatches = 0
for character in goal, state:
    if goal character does not match state character:
        num_mismatches += 1
return num_mismatches
```

My “scrabble” heuristic is to sum the Scrabble value of all the goal characters that don’t match the current state’s characters. This value gets smaller for every action that makes another character match, and it prioritizes changing higher value characters first. Therefore, it’ll give a lower bound for the remaining characters and their cost. Because the goal word’s characters will always be chosen regardless of their Scrabble values, the heuristic can’t overstate the true cost because it uses the goal word’s characters.

Pseudocode:

```
value = 0
for character in goal, state:
    if goal character does not match state character:
        value += scrabble_value(goal character)
return value
```

My “frequency” heuristic is to calculate the path cost using the number of mismatched characters and the minimum rarity in the dictionary. This gives the lower bound of the remaining cost because the minimum number of actions is greater than or equal to the number of mismatched characters, and the rarity is greater than or equal to the minimum rarity. Because the rarity used is the minimum rarity in the entire dictionary, the heuristic can’t overstate the true remaining cost.

Pseudocode:

```
return num_mismatches * (1 + min(dictionary.values()))
```

Question 2.

Word pair	Cost Measure	UCS cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction
WHY -> ASK	Steps	5	773	5	16	97.93%
WHY -> ASK	Scrabble	9	765	9	13	98.30%
WHY -> ASK	Frequency	38.855	808	38.855	754	6.68%
BEAR -> DUCK	Steps	4	691	4	6	99.42%
BEAR -> DUCK	Scrabble	11	2981	11	6	99.80%
BEAR -> DUCK	Frequency	38.24	1084	38.24	714	34.13%

1. UCS and A* did return the same optimal cost for every word pair and every cost measure.
2. For WHY->ASK and for BEAR->DUCK, A* produced the largest reduction for the Scrabble cost measure, at 98.30% and 99.80% reduction respectively.
3. I’m assuming least informative means it produced the minimal reduction in expanded nodes. For WHY->ASK and for BEAR->DUCK, the heuristic was least informative for the Frequency cost measure, at 6.68% and 34.13% reduction respectively.

4. The differences are because the Scrabble heuristic uses the letter value, whereas the Frequency heuristic uses the word's rarity. When comparing the path to the goal, the Scrabble heuristic can use information about the goal to get a better $h(n)$, whereas the Frequency heuristic uses the minimum frequency value to maintain admissibility. The Scrabble heuristic is better informed.

Question 3.

For "steps":

1. Yes, the optimal cost is the same
2. This is because individual path costs are always 1, so the total cost from W_1 to W_2 must be the same as W_2 to W_1 .

For "Scrabble":

1. No, the optimal cost will not always be the same both ways.
2. W_1 and W_2 will not always have the same total Scrabble values, and this means that the best actions with the lowest Scrabble cost will not be the same.
3. DOGCAT(why, ask, scrabble) returned (why shy sha aha ala als ass ask) with cost 11.
DOGCAT(ask, why, scrabble) returned (ask ass ahs aha wha why) with cost 14.

For "frequency":

1. No, the optimal cost will not always be equal both ways.
2. The rarity of the initial word is not considered, so because W_1 and W_2 will not always have the same rarity, this means that the optimal cost will be different even if the paths are reversible.
3. DOGCAT(bear, duck, frequency) returned (bear beak beck buck duck) with cost 38.24.
DOGCAT(duck, bear, frequency) returned (duck buck beck beak bear) with cost 36.79.

Question 4.

1. I'm using Gemini 3.6 Thinking with my UMBC email. Its proposed heuristic was based on the Hamming distance between the current state and the goal, the rarity of the goal word, and the mandatory minimum step costs prior to reaching the goal.

2. I screenshotted what Gemini said was the heuristic. The bottom part is the formula for $r_{\text{step}}(n)$.

We propose the rarity-aware heuristic $h(n)$, defined for any state $n \in D$ as:

$$h(n) = \begin{cases} 0 & \text{if } n = g \\ H(n, g) + \text{rarity}(g) + r_{\text{step}}(n) & \text{if } n \neq g \end{cases}$$

where $r_{\text{step}}(n)$ accounts for mandatory minimum step costs prior to reaching the goal:

$$= \begin{cases} 0 & \text{if } H(n, g) = 1 \\ \min_{u \in \text{Neighbors}(n) \cap D} \text{rarity}(u) + (H(n, g) - 2) \cdot \min_{w \in D} \text{rarity}(w) & \text{if } H(n, g) \geq 2 \end{cases}$$

3. Since the formula used will always use the rarity of both the goal and the mandatory minimum step costs, the valid paths will always give lower bounds greater than or equal to $h(n)$. Therefore, $h(n) \leq h^*(n)$ for any state n .
4. From the given formula, $h(n)$ will never overestimate the true optimal cost because it's using the minimum rarity value. I can verify that the proposal is valid because I also figured out using the minimum rarity value means the rarity can't be overestimated ever.
5. I edited dc.py to add a 'llm' case in self.cost. I did not add 'h0' because my 'steps' heuristic used the number of mismatched characters, which is the same as the "number of positions in which state n differs from the goal." Because the 'llm' case has a higher number of expanded nodes than the 'h0' case, the evidence does not support using the proposed heuristic.

Word pair	Cost Measure	UCS cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction
WHY -> ASK	Steps/h0	5	773	5	16	97.93%
WHY -> ASK	llm	38.855	808	38.855	497	38.49%

BEAR -> DUCK	Steps/h0	4	691	4	6	99.42%
BEAR -> DUCK	llm	38.24	1084	38.24	201	84.46%

Additional Sources:

I used Google and therefore Google Gemini to remind myself of some Python documentation, like how to compare two strings, creating a formatted string, how to properly write function parameters when “self” is involved, how to raise errors, and how to access certain parts of a string in making a new string. With these questions, I made sure to be general, asking for “how to raise an error” and not “how to raise an error if two strings are not equal length.” I also looked at websites like StackOverflow or Python.org instead of copying whatever Gemini spat out. I only mention Gemini because it automatically generates an answer even if I scroll past it.